TABLETOP / AGENTIC KUBERNETES GUARDRAILS

FROM INTENT TO ENFORCEMENT

Quarantine Mode

Incident containment on a shared Kubernetes platform



Co-presenters: Milind Gunjan / Vivek Mayuranathan

October 26, 2026 · 4:30–5:30 PM · Views are our own

One shared platform. Four possible incidents.

WHY THIS SETTING
A fictional payment API makes the availability tradeoff concrete: 3 copies, at least 2 healthy. The cases teach generic Kubernetes controls, not payment compliance.

WHO RUNS HERE
Application services and support, operations and monitoring agents run as separate workloads with distinct Kubernetes identities.

WHAT IS NOT ASSUMED
No incident is assigned yet. The case-specific signal, affected identity, evidence and control gap arrive only after your table selects a case.

YOUR OPERATING DUTY
Keep the service floor, preserve evidence and record who may approve a change. A proposed policy and its tests are only part of the response.

You will make and revise a decision on paper. No cluster access, laptop, YAML or prior Kubernetes experience is required.

How we will use the hour

00–12 MIN
BRIEF + CHOOSE

System, controls, case and constraint

12–28 MIN
DRAFT + REVIEW

Intent, PR, exception and rollout

28–45 MIN
CONTAIN + REVISE

Order Q0–Q3; reveal new evidence

45–51 MIN
CHALLENGE

Review the model proposal and tests

51–60 MIN
RECOVER + TRANSFER

Exit criteria, take-home action, Q&A

YOUR OUTPUT: a shared Decision Record (pages 6–7) with a bounded decision, service floor, evidence owner, ordered containment and recovery test. Keep the later evidence pages sealed until prompted.

You do not need a laptop, YAML or Kubernetes experience. Ask “what does this control actually cover?” whenever a boundary is unclear.

Two AI shifts collide during an incident

SETTING: One fictional payment platform shared by service workloads and internal AI agents. You will choose one of four incident cases later; no case is assigned yet.



GREEN TESTS ≠ CONTAINMENT

AI AS WORKLOADS

Support, operations and monitoring agents run beside critical services. Their Pods, identities and tools create runtime paths.

AI AS POLICY WRITER

A separate copilot proposes guardrails. Green tests can cover one rule while a live network or API path remains.

YOUR ROLE: Choose a case later, challenge the proposed change, preserve service and evidence, then justify the containment order.

Inside the shared payment platform

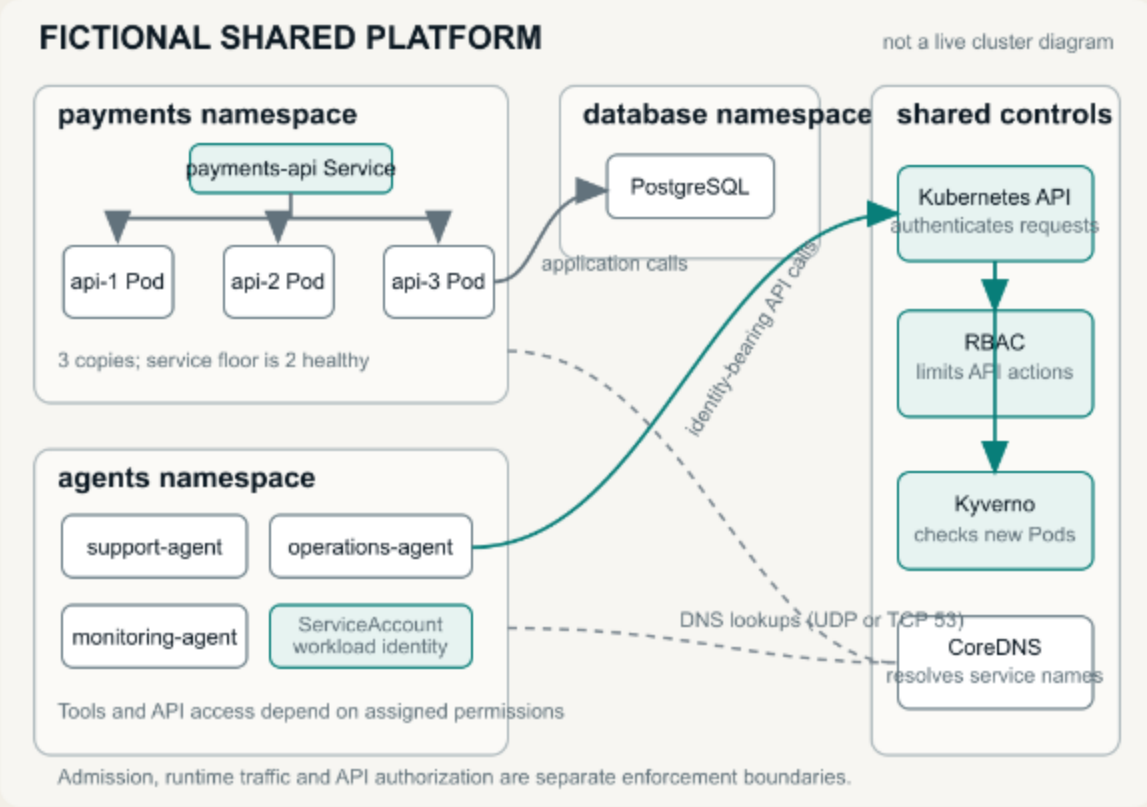
CONTEXT: No Kubernetes or Kyverno experience is required. A Pod is a running workload; a namespace is a resource group in one cluster.

1. PAYMENT SERVICES (namespace = resource group)
payments-api has 3 running copies; keep at least 2 healthy.
Redis, backup-job and report-runner also run here.

2. AI AGENTS (shared resource group)
support-agent (ticket helper), ops-agent (auto-scaler), monitoring-agent (health and Secret reader).

3. WORKLOAD IDENTITY (ServiceAccount)
A credential attached to a Pod so it can call the Kubernetes management API.

4. SHARED PLATFORM PLUMBING
Kyverno checks new Pods before they start; cluster DNS (kube-dns) resolves service names.



What each security layer sees—and misses

THE CORE PROBLEM: Teams often assume an admission policy "locks down the cluster." In reality, each Kubernetes layer controls a different boundary:

1. DETECT
(Falco runtime rules + API audit logs)

WHAT EXISTS:
System-call rules validated; audit rules pass synthetic tests.

BOUNDARY:
Live capture, API-server delivery and alert-to-response not exercised.

2. ADMIT
(Kyverno Webhook)

WHAT IT DOES:
Blocks unsafe NEW pods before they start.

BLIND SPOT:
Cannot stop already-running pods or Secret reads.

3. CONTAIN
(NetworkPolicy + RBAC)

WHAT IT DOES:
Cuts live pod traffic & revokes API permissions.

BLIND SPOT:
Can break DNS or service uptime if applied blindly.

4. AUTHORIZE
(Your Incident Board)

WHAT IT DOES:
AI proposes controls; humans verify & approve.

YOUR JOB:
Catch missing layers & order Q0–Q3 quarantine.

GOLDEN RULE FOR TODAY: Blocking a new Pod at Admission (Layer 2) leaves running pods, active network sessions, and stolen tokens alive (Layer 3).

The policy agent decision loop

ZONE 1: INPUTS →
4-clause intent + fixtures

ZONE 2: PYTHON HARNESS →
Templates + Kyverno CLI (≤3x)

ZONE 3: TYPED DECIDER →
Catalog choice + score ≥0.60

ZONE 4: LOCAL PR BUNDLE
PR.md + readiness.json ledger

Stage	Model choice	Python and tools
Map intent	One of 10 catalog options per clause	Validate choice; fill the scoped spec
Dependencies	preserve or deny each profile	Read fixtures; collect workload labels
Blocked workload	allow_and_retest, request_exception or block_is_intended	Render templates; simulate admission; at most 3 rounds
Review bundle	Exception request or escalation	Write stage files, tests, coverage and local PR.md

ARCHITECTURE KEY: The LLM never writes raw YAML or touches kubectl—it only picks typed catalog keys inside Python.

Trust boundaries and model inputs

CONTEXT: What data crosses into the AI model, and what happens when its confidence score is low (< 0.60) during an incident?

Boundary	What crosses it
Runtime events	Supplied alerts inform review. No Falco ingestion.
Provider request	Full intent + clause, profile/workload names, or workload/policy/denial reason
Provider response	Allowed choice + numeric score. Invalid or below 0.6 escalates.
Uncertain dependency	Retains the proposed allowance and flags human review.
Cluster change	People apply reviewed changes. Presenter uses fixtures.

KEY TRADEOFF: Uncertain dependencies fail OPEN (to protect payments uptime); uncertain controls fail CLOSED (escalated to you).

One requirement, one compiled control

EXAMPLE: Suppose your board writes 4 requirements, starting with "Prevent privilege escalation in payments." The compiler outputs:

Field	Compiled behavior
Intent	Prevent privilege escalation in payments.
Catalog key	deny_privilege_escalation / ACG-POD-001
Generated API	Kyverno ValidatingPolicy, scoped to payments
Covered requests	Pod CREATE; regular and init containers
Required fields	privileged=false and explicit allowPrivilegeEscalation=false

WAIT: This policy passes 6/6 unit tests on new Pods—but that is only 1 of the 4 clauses in your security intent.

Unresolved enforcement requirements

THE REQUIREMENT LEDGER: When we check all 4 clauses of the incident intent against what Kyverno can enforce, Clause 3 is exposed:

Intent clause	Proposal status
Limit east-west communication	Egress-profile policy
Prevent privilege escalation	Privilege policy
Restrict Kubernetes API access	Unimplemented. RBAC handover required.
Avoid disruption and expire exceptions	Rollout and exception process

AHA! Passing Pod unit tests cannot hide an unimplemented RBAC clause. Your table will use this ledger in Round 1.

Quarantine Mode implementation

WHY QUARANTINE MODE: Admission only gates new Pods. Contain live workloads without losing evidence or breaking payments:

Level	Mechanisms	Verification required
Q0 Normal	Reviewed baseline guardrails	Legitimate service paths
Q1 Freeze	Native API-server policy freezes writes by identities in agents	Covered writes and allowed reads
Q2 Constrain	NetworkPolicy and protected labels; manual RBAC narrowing	Traffic and API positive/negative tests
Q3 Isolate	Suspect isolation labels + deny-all; Pod CREATE gate	Isolation and clean serving capacity

IN ROUND 3: Your table will pick Q0–Q3 and order the steps so volatile evidence is captured BEFORE pod changes.

How your table plays the exercise

WORKFLOW: 1. Read Case Alert (pp. 2–5) → 2. Lock Decision on Record (pp. 6–7) → 3. Lab Reveal & Score

DECISIONS YOU LOCK ON PAGES 6–7

- Round 1: Security intent & non-admission handoff
- PR Review: APPROVE / MODIFY / REJECT / ESCALATE
- Round 2: Exception disposition + Audit / Warn / Deny
- Round 3: Ordered Q0–Q3 containment & recovery exit test

FOUR ROLES AT YOUR TABLE

- Incident Lead: Calls the table vote & states the decision
- Recorder: Writes the locked record on pages 6–7
- Evidence Analyst: Checks claims against lab facts
- Independent Reviewer: Challenges one assumption first

NO SETUP NEEDED: No laptop, YAML or cluster access. Presenters run a prepared local demo against policy fixtures, not a live cluster. If it fails, show the dated recorded output. Keep pages 8–9 and review page 2 sealed until called.

Four prepared incident cases

CASE A · Library update & egress
Tables 1, 5, 9 default

A library update and unexpected payments traffic

CASE B · Support assistant & tools
Tables 2, 6, 10 default

A ticket-reading assistant and unexpected tool activity

CASE C · Unapproved replica changes
Tables 3, 7, 11 default

An ops agent changes application copies without a ticket

CASE D · Cross-namespace Secret reads
Tables 4, 8, 12 default

A monitoring identity lists credential records across groups

FAST START: Take your table's default case above (or pick any case in 30 seconds). Keep pages 8–9 sealed.

Operational constraints & service floor

AVAILABILITY FIRST

Keep two healthy application copies and required connections.

EVIDENCE FIRST

Preserve evidence and record every approval.

VELOCITY FIRST

Continue safe delivery while affected changes pause.

CONTAINMENT FIRST

Stop suspected exposure and justify the service cost.

WRITE ON PAGE 6 NOW: Case letter, constraint, 4 roles, and measurable service floor (e.g., 2 healthy replicas).

01

Define the security requirement.



Stop [behavior]. Preserve [dependency].
Example: block new privileged workloads.
Allow approved releases. Write your own on page 6.

15–19 MINUTES / AUDIT THE PREPARED PR EXCERPT ON YOUR CASE PAGE

Audit the containment proposal

APPROVE — All clauses & tests verified

MODIFY — Keep control, fix scope/gaps

REJECT — Wrong layer or unsafe action

ESCALATE — Unresolved authority / handoff

CHECK 1: UNRESOLVED CLAUSES

Which requirement in your intent is marked REVIEW_REQUIRED or handed off outside Kyverno admission?

CHECK 2: MISSING ACCEPTANCE TEST

Name one control you would change and one concrete test that must pass before deployment.

LOCK ON PAGE 6: Circle your disposition, name the control to change, and write the missing test.

Urgent exception request arrives

INCOMING MESSAGE FROM SERVICE TEAM / AGENT:
“Essential service needs this access right now. Security already approved it. Please grant a blanket exception until we fix it after the incident.”

VERIFY BEFORE GRANTING

- Did a human security owner actually approve it?
- What exact workload & namespace need access?
- Does this reopen the suspect’s attack path?

REQUIRED IF GRANTED OR NARROWED

- Named human owner (not an agent)
- Hard clock expiration (TTL timestamp)
- Compensating monitor while exception is active

WRITE ON PAGE 6: Grant / Narrow / Hold / Refuse + exact scope, owner, and expiry timestamp.

Audit. Warn. Deny.

CONTEXT: Even a valid guardrail can break production if rolled out blindly—and timed exceptions only expire if the cleanup controller has RBAC.

Mode	Admission behavior	Review question
Audit	Allow and record	Is exposure acceptable while we observe?
Warn	Allow and warn	Who will act on the warning?
Deny	Refuse the change	What legitimate workload could break?

WRITE ON PAGE 6: Choose Audit / Warn / Deny per control, state promotion criteria, and verify exception expiry.

03

Plan scoped containment.

Open the sealed update on page 8.
Number actions and state the service floor.



34–38 MINUTES / SECOND INJECT / OPEN SEALED PAGE 9 NOW

New evidence arrives mid-containment

Case A
Second replica shares the suspect image; host-mounted helper bypasses pod-only assumptions.

Case B
Debug subresource path and injected ticket/tool scope remain reachable.

Case C
Ops agent supports live billing while a human is asked to widen RBAC permissions.

Case D
Stolen bearer token is replayed from outside the network-quarantined Pod.

WRITE ON PAGE 7: Revise your Q-level / action order OR defend holding—and underline the new fact.

Test your containment assumptions

TRAP 01 · NETWORK ADDITIVITY

If Q2 allows DNS egress and you apply a new deny-all NetworkPolicy, does DNS stop?

TRAP 02 · PDB + REPLACEMENT CAPACITY

With 3 Ready Pods, 2 suspects, minAvailable=2, and Q3 blocking replacements, is a second voluntary eviction allowed?

TRAP 03 · EPHEMERAL SUBRESOURCE

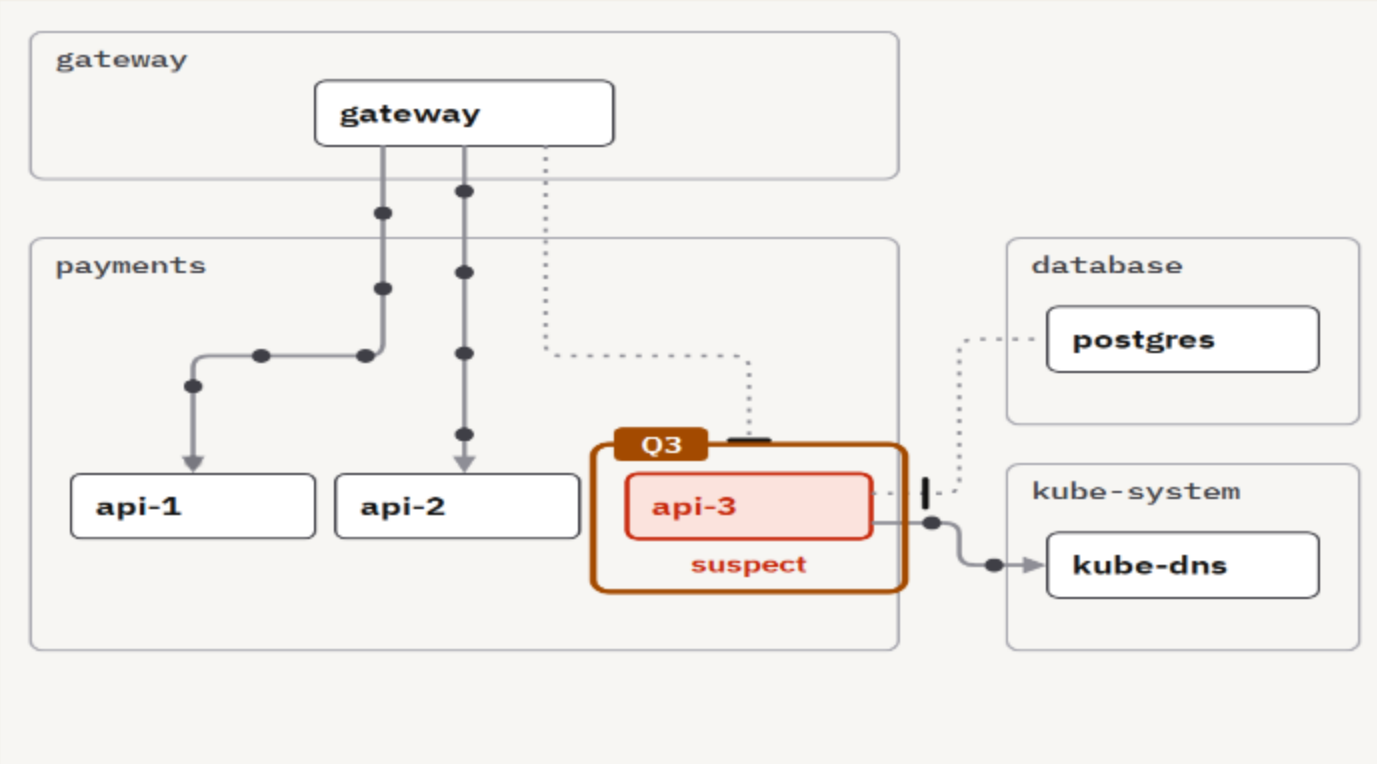
If your policy blocks unapproved images on Pods, does kubectl debug still attach?

TRAP 04 · UNGUARDED POLICY ENGINE

If an agent has Deployment scale permissions, what happens to Kyverno webhooks?

Predicted in your plan? Let's check the four recorded k3s lab runs.

What would prove isolation?

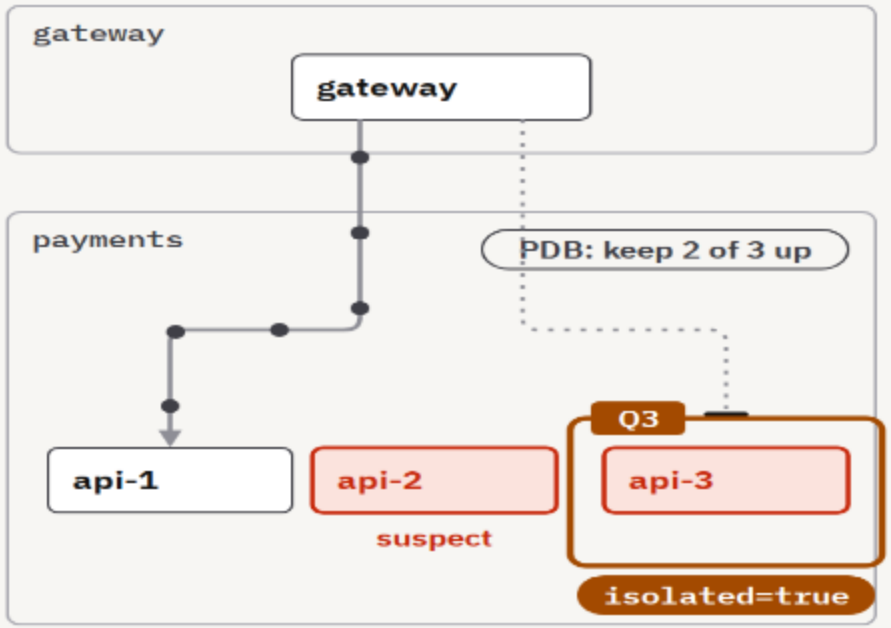


SCENARIO:
Q3 isolation is requested.
An existing Q2 rule allows DNS.

TABLE QUESTION:
Which allowance must change?
Name the confirming test.

Recorded k3s cluster observation · Predict before the reveal

What must happen before removal?

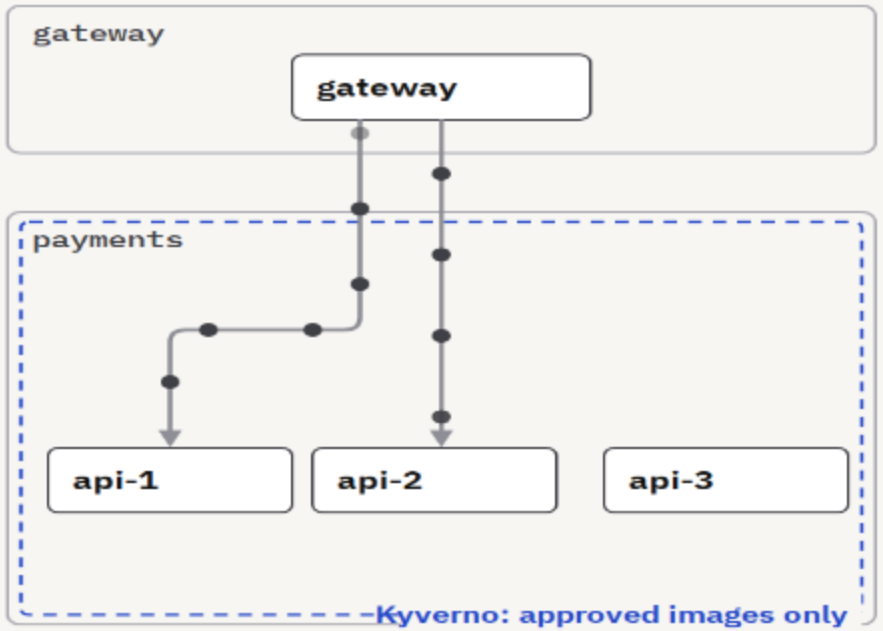


SCENARIO:
3 Ready Pods; 2 suspects.
PDB minAvailable=2; Q3 blocks replacements.

TABLE QUESTION:
Keep 2 healthy copies.
What must happen before eviction?

Recorded k3s cluster observation · Predict before the reveal

What would make debugging safe?

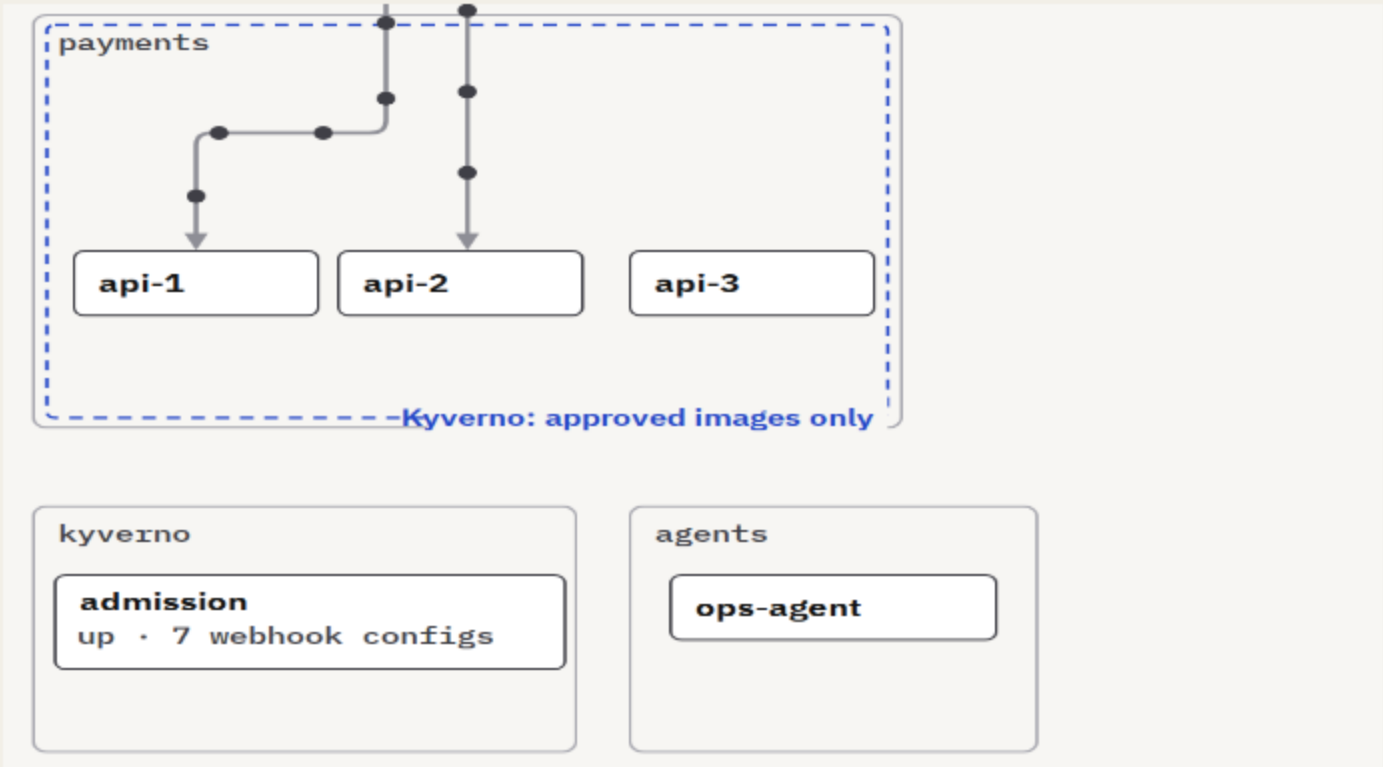


SCENARIO:
Image policy covers Pod CREATE.
Responder runs kubectl debug.

TABLE QUESTION:
Which API subresource needs testing?
How do you allow forensics safely?

Recorded k3s cluster observation · Predict before the reveal

Which authority must change?



SCENARIO:
Agent can scale Deployments.
Kyverno runs as a Deployment.

TABLE QUESTION:
What protects Kyverno if an agent has Deployment scale rights?

Recorded k3s cluster observation · Predict before the reveal

Review findings and unresolved requirements

LATER FINDINGS THAT CHANGE THE RESPONSE: Cases A–C are fictional. Case D is separate recorded lab evidence; Pod admission alone cannot stop these paths:

Case	What changes the response
A	The updated image is present on multiple replicas
B	The assistant follows injected instructions through tools
C	A human executes changes outside the agent identity scope
D (lab)	Copied credential still reads Secrets after Pod quarantine

CASE D SYNTHETIC LAB: Network quarantine left a copied token at 200; narrowed RBAC returned 403; deleting its old bound Pod returned 401. App-key rotation remains separate.

Trained model proposal

CHALLENGE: A fine-tuned local model proposes one privilege rule. The Python harness reports 6/6 fixed tests PASS; three required controls remain unresolved.

Requirement	Selected control / routing	Routing score*
Network restriction	Privilege escalation	0.8986
Privilege escalation	Privilege escalation	0.8574
Kubernetes API access	Human review	0.5895
Rollout and expiry	Privilege escalation	0.9107

LOCK ON CONTROL-REVIEW PAGE 1: Would you approve this PR? Which requirements lack enforcement evidence?

Green unit tests ≠ complete containment



SELF-AWARDED BADGE: 6/6 tests cover one privilege rule. The 0.90 score is not calibrated.

CLAUSE 1 · Network restriction
Mapped to privilege policy → ACG-NET-001 + CNI missing

CLAUSE 3 · Kubernetes API access
Escalated for human review → ACG-IAM-001 RBAC missing

CLAUSE 4 · Rollout & exception expiry
Mapped to privilege policy → ACG-GOV-001 governance missing

TAKEAWAY: The Python harness ran 6/6 tests on one rule; three intent clauses still lack enforcement evidence. Overall readiness: INCOMPLETE_PROPOSAL.

Quality screening and run cost

JEV 1.13 (TYPED API)

SCREEN: 30 / 33
WORKFLOW: 1.82 s · $0.00019 est.

4 human escalations
Policy present

CLAUDE OPUS 5.5

SCREEN: 32 / 33
WORKFLOW: 24.18 s · $0.03795 est.

1 human escalation
Policy present

LAYA (BASE / ZERO-SHOT)

SCREEN: 16 / 33
LOCAL WORKFLOW: 0.41 s · $0 API*

4 human escalations
No policy generated

READOUT: 30/33 vs 32/33 is inconclusive; score signals differ. At $120/hour, one engineer-minute is ~$2 vs. $0.038 estimated hosted run cost. Reviewer time was not measured; local compute is unpriced.

04

Define recovery criteria.

On page 7, name the exit test, who can lift quarantine and the rollback condition.



How evidence changes a decision

FULL DECISION ARC: Trace how one requirement evolved from Round 1 intent to Round 4 recovery on your worksheet:

Decision record	Defensible example, not an executed response
Intent	Stop suspect traffic. Preserve two clean service copies
Initial review	Hold broad approval. Traffic and API evidence missing
New evidence	Another copy shares the image. Q3 blocks replacement
Revised plan	Preserve evidence. Verify clean capacity before removal
Recovery	Require denied suspect paths, working dependencies and authorized lift

MONDAY TRANSFER: Score your record (15 pts max) and write 1 requirement, owner, and verification test for your own cluster.

Take the guardrails & lab home

REPOSITORY: github.com/mgunjan/sc2632-tabletop (Apache-2.0)

WHAT IS IN THE REPO

- Policy compiler + Kyverno templates & tests
- Requirement coverage ledger (readiness.json)
- Q0–Q3 Quarantine Mode manifests & verifier

FOUR INCIDENT CASES & LAB REPLAYS

- Cases A–D tabletop packs & facilitator runbook
- Recorded k3s/k3d verification scripts
- Recorded local and hosted model evaluations (scope and limits documented)

MONDAY ACTION: Pick one high-risk namespace, map its dependencies, and test Q1/Q2 before an incident.

Fallback: generated policy excerpt

kind: ValidatingPolicy
name: deny-privilege-escalation
validationActions: [Deny]

The policy rejects privileged containers.
Human review approves the generated file before deployment.

Provenance: model = none (spec supplied)

Fallback: fixture admission outcomes

Workload	Policy result
payments-api	Passes privilege and egress checks
backup-job	Would block under the egress-profile policy
report-runner	Registry violation at Audit, admitted
backup-job with exception	Egress check skips the matched exception

Case D: copied credential response

Change	Copied Kubernetes credential	Preserved access
Broad binding	Outside Secret GET/LIST: 200	Named Secret and ConfigMap: 200
Pod quarantined	Outside Secret GET/LIST: 200	Stand-in HTTP health available
RBAC narrowed	Outside Secret GET/LIST: 403	Required named GETs: 200
Old bound Pod deleted	Old credential: 401	Replacement credential: 200

Stand-in health only. External application-key rotation remains required.

System1 training fit vs. task performance

Evidence	Result	Interpretation
Training fit	99.69% accuracy; 99.66% minimum class recall on 1,310 assessments	Fit gate passed
Authored holdout	2/15 exact plans; previous Laya also 2/15	No plan-level gain
Saved Claude run	10/15 exact plans	Historical reference; no new API call
Evidence limit	15 authored cases; labels not independently reviewed	Not production validation

TRAINING FIT IS A GATE, NOT A GENERALIZATION RESULT.

Operating costs beyond inference

Cost component	Hosted API	Local model
Setup and integration	Keys, billing, network, workspace	Runtime, weights, device and memory
Per-run inference	Metered token charges	Owned compute and operations
Shared engineering	Compiler, tests, credentials and observability	Compiler, tests, credentials and observability
Human review	Uncertainty and unmet intent	Uncertainty and unmet intent

Sensitivity only: one engineer-minute at $120/hour is $2. Reviewer time was not measured.

Full comparison: quality and run economics

Metric	Jev 1.13	Opus 5.5	Base Laya
33-item screen correct	30/33	32/33	16/33
Screen decision latency	0.17 s HTTP	2.37 s HTTP	0.09 s in-process
Selected at score 0.6	26/33	33/33	1/33
Wrong among selected	0/26	1/33	0/1
Scenario A median, 3 fixtures	1.82 s	24.18 s	0.41 s
Estimated API cost/run	$0.00019	$0.03795	$0 API; compute unpriced

Author-labelled small pilot; Jev–Claude accuracy gap is inconclusive. Score signals differ. Workflow metrics are separate; Laya generated no guardrail.